Multi-Agent Retrieval as an Alternative to Classical RAG: A Controlled Comparison on Educational Tutoring Benchmarks

Anonymous Authors

Abstract— Classical retrieval-augmented generation (RAG) assumes a retrieve-then-read controller, but educational tutoring often depends on personalized learning state and grading criteria rather than the surface query alone. This paper asks whether a multi-agent coordination policy can replace that retrieve-first controller while keeping the downstream tutor fixed. We compare four retrieval mechanisms under matched infrastructure with a shared open-weight tutor/critic backbone. In the released system, routing is computed from adapter-normalized sample fields, and the hybrid pipeline then uses planner, diagnoser, and rubric modules to assemble tutoring context before optional retrieval. On TutorEval and on the matched-sample intersection of EduBench, the proposed pipeline improves the fixed tutor's quality metrics over classical RAG while using fewer tokens; on EduBench it also reduces latency and backbone-call count, showing that the gains are not explained by spending more backbone LLM invocations. Replication on a Qwen3.5-27B-FP8 backbone preserves the efficiency advantage and retains the EduBench quality gains, while TutorEval approaches a quality ceiling for both methods. The result is a narrower but practical conclusion: in educational tutoring, changing how context is assembled before generation can outperform retrieve-first control.

I. Introduction

Classical retrieval-augmented generation (RAG) has become the default mechanism for grounding language-model outputs in external evidence [1]. Its control pattern is fixed by design: the user query is embedded, top-k passages are fetched from an index, and a reader conditions on those passages to generate an answer. This is a strong fit for factual question answering, where the needed evidence is largely determined by the surface query. Educational tutoring is different. In this paper, a tutor is the downstream LLM-driven agent that produces the final educational response: it may explain a concept, diagnose a misunderstanding, score a student answer, or give supportive next-step feedback. Those tutoring decisions often depend on personalized learning state: the student-specific information that should condition the response, such as current level, goals, preferred explanation style, weak points, recent confusion, and likely misconceptions.

This dependence on personalized learning state is visible in both benchmarks we study; Section III introduces them in detail. TutorEval [2] is chapter-grounded science tutoring, whereas EduBench [3] includes explicit profile-conditioned educational scenarios such as Personalized Learning Support (PLS) and Personalized Content Creation (PCC). In both settings, two learners can ask for help on the same topic yet require different tutoring actions because their personalized learning state differs.

This is precisely where classical query-only RAG is a weak fit. A retrieve-then-read controller can fetch topic-similar passages from the query, but it has no explicit stage for constructing personalized learning state before retrieval. In TutorEval, the bottleneck is often identifying which misconception or prerequisite gap the tutor should address inside a long chapter. In EduBench, the bottleneck is often reacting to profile fields, grading criteria, or scenario constraints that are already present in the sample. Once such state is manually appended to the query, the pipeline has already moved beyond classical retrieve-then-read.

A natural alternative is therefore to let a small group of specialist modules build a tutoring brief before answer generation. A planner can propose a scoped search intent; a diagnoser can summarize personalized learning state from the prompt and dialogue; a rubric agent can compile what a strong answer should cover; and only then does a retriever (if anyone) consult the corpus. We refer to this family of systems as multi-agent retrieval. Unlike classical RAG, the retriever is no longer the top-level controller; it becomes an optional tool inside a coordination policy that has already identified what the learner needs.

This paper explores whether multi-agent retrieval can replace classical RAG as the grounding mechanism in educational tutoring. We formulate the comparison as a controlled ablation over four retrieval mechanisms that share the same Qwen3.5-4B tutor/critic backbone (reasoning-token budget 512), the same shared corpus, the same reranker, and the same per-call response cap: (i) Classical RAG (retrieve-always, retrieve-then-read), (ii) Multi-Agent (no retrieval), which replaces the retriever with parallel planner, diagnoser, and rubric modules, (iii) Single-Agent (no retrieval) as an ablation floor, and (iv) our Multi-Agent Retrieval mechanism (hybrid_fast), in which planner, diagnoser, and rubric modules prepare context and retrieval is invoked only when the route indicates it is needed. The contrast is deliberate: all four mechanisms consume the same adapter-normalized tutoring sample, but they assemble different upstream context before the shared tutor writes. We do not force equal total system structure per sample; Table I therefore reports observed backbone-call count as an efficiency

Submitted to the 2026 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Single-blind review.

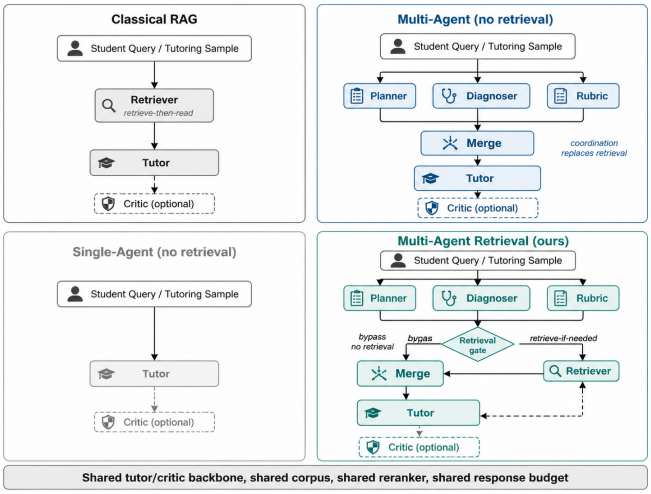

Fig. 1. Forced mechanisms in the controlled comparison, including the proposed hybrid system. Only context assembly and retrieval policy change.

outcome rather than assuming it is constant.

Educational tutoring includes explanation, misconception diagnosis, rubric-based scoring, and next-step feedback. These subproblems require different grounding signals, so a retrieval mechanism cannot be judged on recall alone [4], [5]. On TutorEval (n=834 matched samples) and on the EduBench intersection (n=1562 matched samples), the same tutor scores higher when fed multi-agent retrieval context while using fewer tokens; on EduBench it also reduces latency and backbone calls.

Contributions. (i) We provide a controlled tutoring-side comparison of four retrieval mechanisms under a shared backbone, corpus, reranker, and per-call cap. (ii) We release a training-free router and open framework in which only the context-assembly modules vary while the tutor and critic stay fixed; released configs pin Qwen3.5-4B and Qwen3.5-27B-FP8. (iii) On TutorEval (n=834) and the EduBench intersection (n=1562), paired bootstrap CIs and permutation tests show that multi-agent retrieval beats classical RAG on fixed-tutor quality and efficiency while reducing observed backbone invocations on both benchmarks.

II. Related Work

Our work sits at the intersection of classical RAG, agentic retrieval, and educational tutoring benchmarks. Classical RAG fixes a retrieve-then-read controller [1], while later systems relax that controller through learned or heuristic retrieval decisions such as Self-RAG, CRAG, Adaptive-RAG, and TARG [6]–[9]. Our claim is narrower than novelty over selective retrieval in general: we test a tutoring-specific route-and-context design in which adapter-normalized tutoring samples are routed first and the hybrid pipeline then assembles pedagogical context from planning, diagnosis, and rubric signals before optional retrieval. Evaluation and long-context studies further show that strong retrieval alone does not guarantee grounded generation [10]–[12].

Agentic systems instead treat retrieval as one tool inside a larger coordination loop. ReAct, AutoGen, MA-RAG, and related studies show that role decomposition can improve planning-heavy or staged reasoning tasks [13]–[15]. Our paper makes that comparison explicit for tutoring by holding the downstream tutor fixed and varying only how tutoring context is assembled before generation.

Education-focused evaluation has shifted from short-form accuracy toward pedagogy, adaptivity, and instructional safety. TutorEval, EduBench, and related tutoring benchmarks therefore reinforce the same point: when evaluation measures rubric adherence and calibrated help, a system's grounding mechanism cannot be judged on retrieval recall alone [2]–[5].

III. Datasets

We use two public educational benchmarks with complementary supervision styles. TutorEval [2], released with LM-Science-Tutor, contains over 800 expert-written student-style science questions; its main open-book setting uses all 834 questions, each paired with a long STEM textbook chapter and expert teaching key points. TutorEval is therefore a chapter-grounded tutoring benchmark: the system must explain, disambiguate, and address likely misconceptions while staying aligned with a long chapter rather than a short retrieved snippet.

EduBench [3] is broader. The official benchmark covers nine educational scenarios, split into five student-oriented tasks and four teacher-oriented tasks, with over 4,000 educational situations and 12 evaluation dimensions. These scenarios include question answering, error correction, personalized support, emotional support, grading, teaching-material generation, and personalized content creation. The released rows contain a prompt, scenario metadata, and multiple reference model predictions; our adapter turns those predictions into consensus-style supervision through reference-score statistics and rubric terms. For paper reporting, we use the 1562-example four-way intersection completed by all forced architectures. Personalized-learning signals are especially explicit in PLS and PCC rows, where student profiles specify level, goals, study habits, interests, and weak points.

IV. Methodology

Our proposed framework, Multi-Agent Retrieval, is a training-free tutoring pipeline with route-first control. Its top-level supervisor is a per-sample route object storing the architecture family, pedagogical regime, and retrieval/rubric/critic switches. Each adapter-normalized sample contains the question, optional dialogue, context text, rubric items, and metadata; adapters may inject regime hints or rubric-like fields before routing. In the hybrid pipeline, planner, diagnoser, and rubric modules build the tutoring brief before optional retrieval, while

the downstream tutor and critic stay fixed. Fig. 2 sketches this pipeline.

This separation matters because the two benchmarks stress different failure modes. In TutorEval, the missing information is often chapter-grounded but pedagogically scoped: the tutor must connect the question to the right example, equation, or misconception inside a long chapter. In EduBench, the missing information is often student-specific: the system must react to profile fields, grading criteria, or scenario constraints that are already present in the sample but are not naturally retrievable from a topical query. A retrieve-first controller treats these cases too similarly. Our pipeline makes the tutoring brief explicit before answer generation, even though the top-level route is computed first.

To isolate that design choice, all implemented mechanisms share the same tutor/critic backbone, corpus, reranker, context packer, and response budget; the only designed difference is how supporting context is assembled before the tutor writes. For each benchmark we freeze one common retrieval index and keep its construction unchanged across mechanisms. The design principle is simple: retrieve when external evidence is genuinely needed, and otherwise spend the budget on building a better pedagogical view of the learner and the grading criteria. Backbone calls are therefore treated as an observed efficiency outcome rather than a quantity forced to be equal by design. The head-to-head tables should thus be read as controlled pipeline ablations under shared routing metadata, not as a benchmark of one online router making a four-way architecture choice.

A. Regime-aware routing

The released router is evaluated before any tutoring pipeline runs. The framework can route from a trained bag-of-ngrams classifier, exact dataset-name priors, or the cue-based fallback described below; the reported paper runs use only the latter two because no trained router model is loaded in the released experiment configs. For each adapter-normalized sample x, the fallback constructs a normalized text blob $b(x)$ from the question, optional context, rubric text, and dialogue history, and computes cue-match rates

$$\kappa_G(x) = \frac{1}{|G|} \sum_{u \in G} \mathbb{1}[u \subseteq b(x)], \tag{1}$$

for five cue sets: evidence (E), coordination (C), rubric (R), planning (P), and tutoring (T). Each score follows one common form,

$$s_j(x) = \mathrm{clip}_{[0,1]}(\kappa_j(x) + b_j(x) + \pi_j(d_x)), \tag{2}$$

for $j \in \{e, c, r, p, t\}$, where $b_j(x)$ is a small sample-dependent bonus and $\pi_j(d_x)$ is the dataset prior. In the released configuration, $b_e = 0.08\mathbb{1}[\text{context}] + 0.05\mathbb{1}[\text{image}]$, $b_c = 0.03\mathbb{1}[\text{image}]$, $b_r = 0.20\mathbb{1}[\text{rubric}]$, $b_p = 0$, and $b_t = \min(|h_x|/6, 0.5)$. Dataset priors add $+0.15$ to evidence for evidence-grounded datasets, $+0.15$ to coordination and rubric for rubric-feedback datasets, $+0.15$ to coordination and tutoring for adaptive-tutoring datasets, and $(+0.12, +0.18)$ to coordination and planning for lesson-planning datasets. High s_e means the sample looks evidence-grounded; high s_c, s_r, s_p, or s_t indicate increasingly coordination-heavy tutoring, rubric, planning, or dialogue-sensitive cases. These are fixed training-free heuristics: 0.52, 0.50, 0.35, and 0.45 come from released config defaults, whereas 0.55, 0.28, and 0.42 remain embedded in the router code; none are retuned on the reported evaluation slices.

Architecture selection is then a fixed case split:

$$A(x) = \begin{cases} \text{RAG} & s_e \geq 0.52,\ s_c < 0.35, \\ \text{A-RAG} & s_e \geq 0.35,\ s_c \geq 0.50, \\ \text{MA} & s_c \geq 0.55,\ s_e < 0.28, \\ \text{Hybrid} & \text{otherwise.} \end{cases} \tag{3}$$

This case split describes the fallback router's family choices. High evidence with low coordination maps to classical retrieve-then-read, high evidence with high coordination maps to agentic RAG, and high coordination with little evidence maps to multi-agent tutoring without retrieval. Borderline cases fall into the hybrid family. The paper's Single-Agent (no retrieval) baseline is a separate ablation floor and is not produced by the router. The pedagogical regime is resolved separately. If the dataset adapter provides a regime hint, the router uses it directly; otherwise it sets

$$R(x) = \begin{cases} \text{LP} & s_p \geq \max\{s_r, s_t, 0.42\}, \\ \text{RF} & s_r \geq \max\{s_t, 0.35\}, \\ \text{AT} & s_t \geq 0.35, \\ \text{EG} & \text{otherwise.} \end{cases} \tag{4}$$

where LP, RF, AT, and EG denote lesson planning, rubric feedback, adaptive tutoring, and evidence-grounded reasoning. Within the proposed hybrid family, retrieval is gated by

$$g(x) = \mathbb{1}[s_e(x) \geq 0.35 \ \vee \ R(x) = \text{EG}], \tag{5}$$

which matches the production threshold in the codebase. Inside the intended hybrid pipeline, if $g(x) = 0$, the system still builds the tutoring brief and skips retrieval unless no chunks have yet been retrieved and $s_e(x) \geq 0.45$, which triggers one fallback retrieval pass on the raw question. The reported forced comparisons are slightly different: the app first computes the full route and then overwrites only the architecture label, reusing the already computed retrieval, regime, rubric, and critic flags. Hybrid TutorEval should therefore be interpreted as coordination layered on always-on retrieval inherited from shared route metadata, not as repeated online firing of $g(x)$.

B. Specialist agents and shared context

All implemented mechanisms operate over the same tutoring prompt, inferred learning state, rubric summary, retrieved evidence, and draft answer. The tutor and the critic are held constant across all implemented

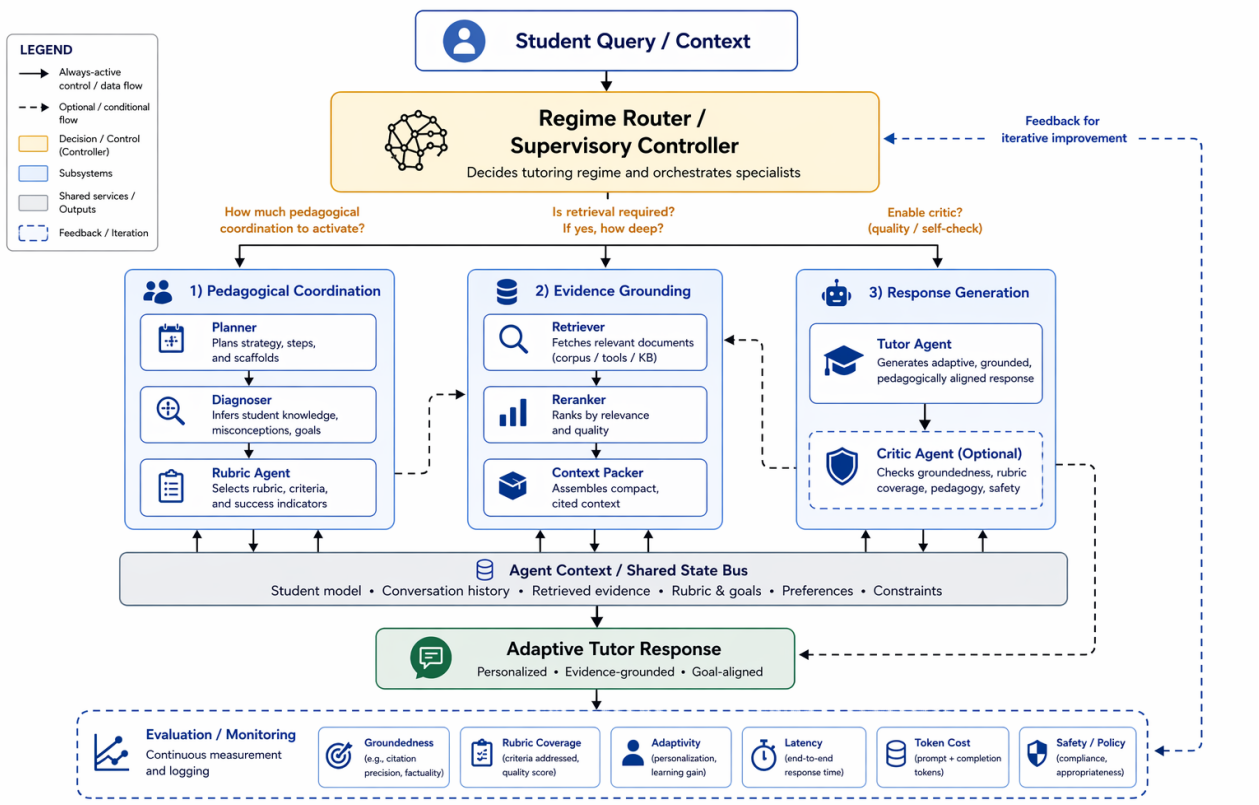

Fig. 2. Main framework of Multi-Agent Retrieval. In the released implementation, the router scores the adapter-normalized tutoring sample first; inside the hybrid pipeline, planning, personalized-learning-state diagnosis, and rubric summarization then assemble the tutoring brief before optional retrieval, with one fallback pass only when no chunks were retrieved and $s_e \geq 0.45$.

mechanisms: the tutor writes from the supplied context, and the route-controlled critic may revise the draft; on TutorEval and EduBench it is usually enabled because both adapters stamp adaptive-tutoring regime hints. The only moving parts are the upstream context builders. In the released paper configuration, planner, diagnoser, and rubric are deterministic heuristics. Their roles are asymmetric. The planner alone is retrieval-facing: it emits up to three deduplicated queries from the raw question, extracted key terms, dataset cueing, and the first rubric items. The diagnoser and rubric agent are tutor-facing: they summarize transient learning state and answer criteria for the tutor prompt, but they do not alter retriever scoring. In this release, learning state is inferred from the current question and dialogue rather than from a persistent profile store.

The tutor itself is dataset-aware but architecturally fixed. Its prompt includes the dataset name, question, recent dialogue, inferred learning state, operating plan, rubric summary, and either retrieved evidence or inline benchmark context. It is instructed to be correct, pedagogically useful, concise, and to end with a next step or check-for-understanding when appropriate. Retrieved evidence must be cited with [doc_id] markers. EduBench consensus rows require a JSON object with a score, scoring details, and personalized feedback; TutorEval and the other tutoring-style profiles use instructional prose. This design isolates the effect of changing the retrieval context while leaving the downstream writer unchanged; personalization reaches retrieval mainly through planner query formulation, not through profile-aware reranking.

C. Hybrid retrieval, reranking, and context packing

The retrieval stack is intentionally lightweight and also follows the codebase exactly. For a query q and chunk c, the hybrid index computes

$$\begin{aligned}\text{score}_0 = 0.58\, k_w + 0.17\, k_{ch} \\ + 0.25\, k_\ell + \beta,\end{aligned} \tag{6}$$

where all terms are computed for the current (q, c) pair. Here k_w is word-level TF–IDF similarity, k_{ch} is character 3–5 gram TF–IDF similarity, and k_ℓ is the similarity in the truncated-SVD latent space. The query boost β adds 0.01 per overlapping title token and, in citation-seeking mode, an additional 0.025 for chunks whose source type is reference, lecture, or document. When multiple planner queries are issued, the retriever merges all returned candidates and deduplicates them by chunk identifier before reranking.

In the released paper configuration, no trained reranker is loaded, so the heuristic reranker score is

$$\begin{aligned}\text{score}_1 = 0.55\,\text{score}_0 + 0.20\, o_t + 0.10\, o_h \\ + 0.06\, m_p + 0.05\, o_n + 0.04\, \rho,\end{aligned} \tag{7}$$

where all terms are again computed for the current (q, c) pair; o_t, o_h, m_p, and o_n denote token overlap, title

overlap, exact phrase match, and numeric overlap, and ρ is a brevity prior. Once planner queries are merged, reranking remains question-centric and does not consume the learner summary directly. Final context packing uses maximum marginal relevance:

$$c^\star = \arg\max_{c \in \mathcal{C} \setminus S} \lambda \, \mathrm{score}_1(q, c) - (1 - \lambda) \max_{c' \in S} \mathrm{overlap}(c, c'), \tag{8}$$

with $\lambda = 0.68$, a four-chunk cap, and a 6000-character context budget. This keeps the comparison focused on retrieval quality rather than on simply enlarging the context window.

D. Evaluation protocol

Evaluation is dataset-aware, but the reported metrics are internal automatic proxies implemented in the released evaluator rather than official benchmark scoring scripts. The headline table's composite score (Comp.) is a paper-level weighted summary from the released analysis pipeline, reported only as a convenience overview; we interpret the task-specific metrics directly. Token-F1 is the token-overlap F1 between the model answer and the benchmark reference answer. Rubric coverage is the fraction of rubric items the answer covers, using either near-verbatim phrase match or sufficiently strong token overlap anchored by a content-bearing token. Latency, total tokens, and backbone-call count capture inference cost, and backbone calls correspond to the released system's llm_call_count.

For EduBench, we additionally report EB12D, the mean over 12 rubric-oriented signals spanning scenario adaptation, factual reasoning, and pedagogical application. Those signals reward output-format compliance, supportive but non-negative tone, relevance to the question, use of scenario elements from the sample, alignment with the benchmark score, domain-knowledge grounding, reasoning quality, error identification, clarity, motivational guidance, personalization, and higher-order prompting. We report EduAlign as $1 - |s(y) - \bar{s}|/100$, where $s(y)$ is the score extracted from the model's JSON output and $\bar{s}$ is the benchmark reference score.

For TutorEval, we report tutor-hit as the mean keypoint credit over the K expected teaching points, with full credit for explicit coverage, partial credit for moderate token overlap, and zero otherwise. We also track a chapter-grounding proxy by measuring whether the response stays aligned with retrieved evidence when retrieval is used, or with the inline chapter text otherwise. Concretely, EduBench is evaluated as a structured educational-judgment task, whereas TutorEval is evaluated as a chapter-grounded tutoring-response task. In the per-example evaluator outputs, off-profile metrics remain present as zeros; in the paper tables, benchmark-inapplicable columns are simply not shown. The released code also records a workload audit $U = T + 160Q + 90C + 240A + 30E$, where T is total tokens, Q retrieval queries, C retrieved chunks, A materialized agent outputs, and E trace events. This score is used only as an efficiency audit, not as a training objective. A retrieval-agnostic corpus-factuality check is supported when a compatible shared index is available, but the archived main comparison predates that audit, so we do not present it as a headline result here.

V. Experimental Results

We run frozen head-to-head comparisons under matched 4B infrastructure (Qwen3.5-4B, reasoning budget 512, temperature 0.15, response cap 900). TutorEval uses all 834 matched samples and EduBench uses the 1562-sample four-way intersection; retrieval hyperparameters are fixed at the §IV values.

A. Primary comparison: TutorEval

Table I reports paired matched-sample comparisons with bootstrap confidence intervals and permutation p-values. On TutorEval, changing the retrieval context raises Token-F1 by +0.004, tutor-hit by +0.019, and rubric coverage by +0.026, while using 484 fewer tokens; latency trends lower but is not reliable (p=0.21). In the released forced-comparison harness, TutorEval hybrid still retrieves on every sample because route metadata are computed before architecture forcing. The benchmark therefore tests whether coordination still helps when chapter evidence is always consulted. Retrieval-free multi-agent and single-agent baselines lag, showing that the fixed tutor does not recover chapter-specific key points reliably without corpus support.

B. EduBench intersection and the capacity confound

On the EduBench intersection, multi-agent retrieval improves EB12D by +0.031 and rubric coverage by +0.186, while reducing latency by 5,923 ms, tokens by 1,794, and backbone calls by 0.49. Classical RAG keeps a small EduAlign edge, but retrieval is skipped on every EduBench sample in the frozen run, indicating that these prompts are usually better served by learner- and rubric-centered coordination than by external lookup. The hybrid therefore behaves almost identically to retrieval-free multi-agent coordination on EduBench ($\Delta_{\mathrm{EB12D}} = +0.001$, p=0.61) while clearly outperforming classical RAG.

C. Ablations: attributing the hybrid's gains

Two executed 4B ablations on a shared TutorEval slice (n=100) isolate the mechanism. Forcing retrieval hurts Token-F1 (−0.003), tutor-hit (−0.015), and rubric coverage (−0.025) with little token change (−36), so the gate is functionally active. Turning the shared critic off improves all scalar metrics and cuts tokens by 27% (−891), implying that the hybrid gain does not come from the critic. On a pinned Qwen3.5-27B-FP8 backbone, all eight sessions completed: TutorEval score deltas collapse near zero, but hybrid remains more efficient (−992 tokens, −11,845 ms), and the EduBench pattern repeats on the

TABLE I

Head-to-head results on TutorEval (n=834 matched samples) and the EduBench four-way intersection (n=1562). Higher is better except latency, tokens, and backbone calls. Δ rows report (ours − classical RAG) on matched samples only; best values are bold.

Retrieval mechanism	Comp.	Tok-F1	EB12D	TE Hit	Rubric	EduAlign	Lat. (ms)	Tokens	Backbone calls
TutorEval, n=834 matched samples (Qwen3.5-4B, reasoning budget 512 tokens)									
Classical RAG	0.377	0.112	–	0.938	0.919	–	51 903	3 975	1.84
Multi-Agent (no retr.)	0.371	0.090	–	0.932	0.907	–	51 048	5 959	1.67
Single-Agent (no retr.)	0.360	0.086	–	0.900	0.867	–	55 257	6 466	2.00
Multi-Agent Retrieval (ours)	0.383	0.115	–	0.957	0.944	–	49 113	3 490	1.56
Δ (ours − Classical RAG)	+0.006	+0.004*	–	+0.019**	+0.026**	–	−2,790	−484***	−0.28***
95% CI	–	[0.0003, 0.007]	–	[0.006, 0.032]	[0.007, 0.045]	–	[−7076, 1534]	[−558, −410]	[−0.31, −0.24]
EduBench intersection, n=1562 matched samples (Qwen3.5-4B)									
Classical RAG	0.367	–	0.367	–	0.705	0.166	19 828	4 020	1.97
Multi-Agent (no retr.)	0.398	–	0.398	–	0.889	0.130	13 916	2 216	1.47
Single-Agent (no retr.)	0.377	–	0.377	–	0.700	0.153	19 195	2 976	1.92
Multi-Agent Retrieval (ours)	0.398	–	0.398	–	0.891	0.126	13 904	2 226	1.48
Δ (ours − Classical RAG)	+0.031***	–	+0.031***	–	+0.186***	−0.040**	−5,923***	−1,794***	−0.49***
95% CI	[0.027, 0.036]	–	[0.027, 0.036]	–	[0.174, 0.199]	[−0.069, −0.011]	[−6628, −5242]	[−1845, −1738]	[−0.52, −0.46]

$^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$; CIs use 2000 paired bootstraps and p-values use 10000 permutations.

27B intersection (n=87) with EB12D +0.032, rubric +0.210, tokens −1,861, and latency −22,070 ms.

VI. Limitations

Limitations remain. The study fixes backbone, cap, corpus, and retriever but not the full system structure; it reports automatic rather than human evaluation; and it is limited to two open-weight backbones, one shared corpus per benchmark, English-only data, and benchmark outputs rather than downstream learning gains.

VII. Conclusion

Under shared infrastructure, multi-agent retrieval improves fixed-tutor quality and efficiency on TutorEval and especially on the EduBench intersection. The gain comes from changing how tutoring context is assembled, not from adding more backbone calls. In educational tutoring, better context assembly before generation can beat classical retrieve-first control.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. K"uttler, M. Lewis, W. tau Yih, T. Rockt"aschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[2] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, "Language models as science tutors," 2024. [Online]. Available: https://arxiv.org/abs/2402.11111

[3] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "Edubench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," 2025. [Online]. Available: https://arxiv.org/abs/2505.16160

[4] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "Mathtutorbench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, 2025, pp. 204–221. [Online]. Available: https://aclanthology.org/2025.emnlp-main.11/

[5] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "Tutorbench: A benchmark to assess tutoring capabilities of large language models," 2025. [Online]. Available: https://arxiv.org/abs/2510.02663

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2024. [Online]. Available: https://arxiv.org/abs/2403.14403

[9] Y. Wang, L. Wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient RAG," 2026. [Online]. Available: https://arxiv.org/abs/2511.09803

[10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2408.08067

[11] R. Friel, M. Belyi, and A. Sanyal, "RAGBench: Explainable benchmark for retrieval-augmented generation systems," 2024. [Online]. Available: https://arxiv.org/abs/2407.11005

[12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," Transactions of the Association for Computational Linguistics, vol. 12, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2024. [Online]. Available: https://arxiv.org/abs/2308.08155

[15] T. Nguyen, P. Chin, and Y.-W. Tai, "Ma-rag: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," 2025. [Online]. Available: https://arxiv.org/abs/2505.20096